



ДИСЦИПЛИНА

Разработка приложений на языке Котлин

(полное наименование дисциплины без сокращений)

ИНСТИТУТ

информационных технологий

КАФЕДРА

информационных технологий в атомной энергетике

полное наименование кафедры

ВИД УЧЕБНОГО
МАТЕРИАЛА

Лекция

(в соответствии с пп 1-11)

ПРЕПОДАВАТЕЛЬ

Золотухин Святослав Александрович

(фамилия, имя, отчество)

СЕМЕСТР

5 семестр 2025 – 2026 учебный год

(указать семестр обучения, учебный год)



Лекция 2. Разработка приложений на языке Котлин

ФИО преподавателя: Золотухин Святослав Александрович

e-mail: zolotuhin@mirea.ru



Функции

Формальная запись:

```
fun имя_функции (параметры) : возвращаемый_тип {  
    выполняемые инструкции  
}
```

Параметры необязательны



Функции

Пример функции

```
fun main() {  
    text() // вызов функции text  
    text() // вызов функции text  
}  
  
// определение функции text  
fun text(){  
    println("Какой-то текст")  
}
```



Функции. Передача параметров

Пример функции

```
fun main() {  
    text("Какой-то текст") //вызов функции text с параметром  
    text("Другой какой-то текст")  
}  
  
// определение функции text с параметром  
fun text(info: String){  
    println(info)  
}
```



Функции. Передача параметров

Другой пример функции

```
fun main() {  
    displayUser("Никита", 20)  
    displayUser("Алиса", 19)  
    displayUser("Андрей", 21)  
}  
  
fun printUser(name: String, age: Int){  
    println("Имя: $name  Возраст: $age")  
}
```



Аргументы по умолчанию

```
fun main() {  
    printUser("Andrew", 20, "Programmer")  
    printUser("Kristine", 19)  
    printUser("Jane")  
}  
  
fun printUser(name: String, age: Int = 18, position: String="unemployed"){  
    println("Name: $name  Age: $age  Position: $position")  
}
```

```
↑ C:\Users\svt\.jdk\corretto-17.0.8.1\bin\java.exe  
↓ Name: Andrew, Age: 20, Position: Programmer  
  Name: Kristine, Age: 19, Position: Unemployed  
  Name: Jane, Age: 18, Position: Unemployed
```



Именованные аргументы

```
fun main() {  
    printUser("Andrew", post="Programmer", age=25)  
    printUser(age=20, name="Kate")  
    printUser("Kate", post="Junior Programmer")  
}
```

```
↑ C:\Users\svt\.jdk\corretto-17.0.8.1\bin\java.exe  
↓ Name: Andrew, Age: 25, Position: Programmer  
⇐ Name: Kate, Age: 20, Position: Unemployed  
⇐ Name: Kate, Age: 18, Position: Junior Programmer  
⇐
```




Именованные аргументы

```
fun printUser(age: Int = 18, name: String){  
    println("Name: $name  Age: $age")  
}
```

```
fun main() {  
    printUser(name="Tom", age=28)  
    printUser(name="Kate")  
}
```



Изменение параметров

```
fun foo(n: Int){  
    n = n * 2 // Ошибка - значение параметра нельзя  
    ИЗМЕНИТЬ  
    println("n: $n")  
}  
  
val cannot be reassigned
```



Изменение параметров

```
fun foo(numbers: IntArray){  
    numbers[0] = numbers[0] * 2  
    println("Значение в функции foo: ${numbers[0]}")  
}  
  
fun main() {  
    var nums = intArrayOf(2, 3, 5)  
    foo(nums)  
    println("Значение в функции main: ${nums[0]}")  
}
```

```
↑ C:\Users\svt\.jdk\corretto-17.0.8.1\bin\java.exe  
Значение в функции foo: 4  
↓  
Значение в функции main: 4  
=
```



Переменное количество параметров. Vararg

```
fun printStrings(vararg strings: String){  
    for(str in strings)  
        println(str)  
}  
  
fun main() {  
    printStrings("Joe", "Kate", "Michael")  
    printStrings("Kotlin", "Java", "Python", "C#")  
}
```



Переменное количество параметров.

Vararg

Например, подсчет суммы какого-то количества чисел

```
fun sumNumbers(vararg nums: Int){  
    var result=0  
    for(n in nums)  
        result += n  
    println("Сумма чисел: $result")  
}  
  
fun main() {  
    sumNumbers(1, 2, 3) // Сумма чисел: 6  
    sumNumbers(1, 2, 3, 4, 5, 6, 7, 8, 9, 10) // Сумма чисел: 55  
}
```



Переменное количество параметров.

Vararg

Если функция принимает несколько параметров, то обычно vararg-параметр является последним.

```
fun printUsers(count:Int, vararg users: String){  
    println("Count: $count")  
    for(user in users)  
        println(user)  
}
```

```
fun main() {  
    printUsers(3, "Andrew", "Kate", "Ashley")  
}
```

```
C:\Users\svt\.jdk\corretto  
Count: 3  
Andrew  
Kate  
Ashley
```



Переменное количество параметров.

Vararg

Это необязательно.

```
fun printUsers(institute: String, vararg users: String, count:Int){
```

```
    println("Institute: $institute")
```

```
    println("Count: $count")
```

```
    for(user in users)
```

```
        println(user)
```

```
}
```

```
fun main() {
```

```
    printUsers("IT", "Andrew", "Kate", "Ashley", count=3)
```

```
}
```

```
C:\Users\svt\.jdk\corrett  
Institute: IT  
Count: 3  
Andrew  
Kate  
Ashley
```



Возвращение результата. Оператор return

```
fun subtract(x: Int, y: Int): Int {  
    return x - y  
}
```

```
fun main() {  
    val a = subtract(4, 3)  
    val b = subtract(6, 6)  
    val c = subtract(10, 5)  
    println("a=$a b=$b c=$c")  
}
```

```
C:\Users\svt\.jdk\corretto-17.0.8.1\  
a=1 b=0 c=5  
Process finished with exit code 0
```




Unit

Функция

```
fun foo(){  
    println("Hello, Kotlin")  
}
```

аналогична

```
fun foo() : Unit {  
    println("Hello, Kotlin")  
}
```

val a = foo() // не имеет смысла



Unit

Если функция возвращает значение Unit, можно использовать оператор return для возврата из функции

```
fun printPositiveNumbers(numbers: List<Int>) {  
    for (number in numbers) {  
        if (number < 0) {  
            println("Отрицательное число обнаружено: $number")  
            return // Досрочное завершение выполнения функции  
        }  
        println("Положительное число: $number")  
    }  
}
```



Однострочные функции

Формальная запись:

fun имя_функции (параметры_функции) = тело_функции

fun square(x: Int) = x * x

```
fun main() {  
    val a = square(5)  // 25  
    val b = square(6)  // 36  
    println("a=$a b=$b")  
}
```



Локальные функции

```
fun compareNumbers(num1: Int, num2: Int) {
```

```
    fun isValidNumber(num: Int): Boolean {
```

```
        return num >= 0 && num <= 100
```

```
    }
```

```
    if (!isValidNumber(num1) || !isValidNumber(num2)) {
```

```
        println("Неверное число")
```

```
        return
```

```
    }
```

```
    when {
```

```
        num1 == num2 -> println("num1 == num2")
```

```
        num1 > num2 -> println("num1 > num2")
```

```
        num1 < num2 -> println("num1 < num2")
```

```
    }
```

```
}
```

Локальная функция может использоваться только в той функции, где она определена

```
fun main() {
```

```
    compareNumbers(20, 23) // num1 < num2
```

```
    compareNumbers(-3, 20) // Неверное число
```

```
    compareNumbers(34, 134) // Неверное число
```

```
    compareNumbers(15, 8) // num1 > num2
```

```
}
```



Перегрузка функций

```
fun subtract(a: Int, b: Int) : Int{  
    return a - b  
}  
  
fun subtract(a: Double, b: Double) : Double{  
    return a - b  
}  
  
fun subtract(a: Int, b: Int, c: Int) : Int{  
    return a - b - c  
}  
  
fun subtract(a: Int, b: Double) : Double{  
    return a - b  
}  
  
fun subtract(a: Double, b: Int) : Double{  
    return a - b  
}
```

Перегруженные версии должны отличаться по типу, порядку или количеству параметров



Перегрузка функций

```
fun subtract(a: Double, b: Int) : Double {  
    return a - b  
}  
fun subtract(a: Double, b: Int) : String {  
    return "$a - $b"  
}  
// Ошибка
```



Тип функции

Формальная запись:

(типы_параметров) -> возвращаемый_тип

```
fun printHello(){                                () -> Unit  
    println("Hello Kotlin")  
}
```

```
fun subtract(a: Int, b: Int): Int{                (Int, Int) -> Int  
    return a - b  
}
```



Переменные-функции

```
fun main() {  
    val message: () -> Unit  
    message = ::hello  
    message()  
}
```

```
fun hello(){  
    println("Hello Kotlin")  
}
```




Переменные-функции

```
fun main() {  
    val operation: (Int, Int) -> Int = ::sum  
    val result = operation(3, 5)  
    println(result) // 8  
}  
  
fun sum(a: Int, b: Int): Int{  
    return a + b  
}
```



Функции высокого порядка

Функция как параметр функции

```
fun main() {  
    displayMessage(::morning)  
    displayMessage(::evening)  
}  
  
fun displayMessage(mes: () -> Unit){  
    mes()  
}  
  
fun morning(){  
    println("Good Morning")  
}  
  
fun evening(){  
    println("Good Evening")  
}
```



Возвращение функции из функции

```
fun main() {  
    val action1 = selectAction(1)  
    println(action1(8,5)) // 13  
    val action2 = selectAction(2)  
    println(action2(8,5)) // 3  
}  
  
fun selectAction(key: Int): (Int, Int) -> Int{  
    // определение возвращаемого результата  
    when(key){  
        1 -> return ::sum  
        2 -> return ::subtract  
        3 -> return ::multiply  
        else -> return ::empty  
    }  
}
```

```
fun empty (a: Int, b: Int): Int{  
    return 0  
}  
fun sum(a: Int, b: Int): Int{  
    return a + b  
}  
fun subtract(a: Int, b: Int): Int{  
    return a - b  
}  
fun multiply(a: Int, b: Int): Int{  
    return a * b  
}
```



Анонимные функции

```
fun(x: Int, y: Int): Int = x - y
```

или

```
fun(x: Int, y: Int): Int{  
    return x - y  
}
```

Анонимную функцию можно передавать в качестве значения переменной

```
fun main() {  
    val message = fun() = println("Hello")  
    message()  
}
```



Анонимные функции

Анонимная функция с параметрами

```
fun main() {  
    val subtract = fun(x: Int, y: Int): Int = x - y  
    val result = subtract(5, 4)  
    println(result)    // 1  
}
```



Анонимные функции

Анонимная функция как аргумент функции

```
fun main() {  
    doOperation(9,5, fun(x: Int, y: Int): Int = x + y )    // 14  
    doOperation(9,5, fun(x: Int, y: Int): Int = x - y)    // 4  
    val action = fun(x: Int, y: Int): Int = x * y  
    doOperation(9, 5, action)    // 45  
}  
  
fun doOperation(x: Int, y: Int, op: (Int, Int) ->Int){  
    val result = op(x, y)  
    println(result)  
}
```



Анонимные функции

Возвращение анонимной функции из функции

```
fun main() {  
    val action1 = selectAction(1)  
    val result1 = action1(4, 5)  
    println(result1)    // 9  
    val action2 = selectAction(3)  
    val result2 = action2(4, 5)  
    println(result2)    // 20  
    val action3 = selectAction(9)  
    val result3 = action3(4, 5)  
    println(result3)    // 0  
}  
  
fun selectAction(key: Int): (Int, Int) -> Int {  
    // определение возвращаемого результата  
    when(key){  
        1 -> return fun(x: Int, y: Int): Int = x + y  
        2 -> return fun(x: Int, y: Int): Int = x - y  
        3 -> return fun(x: Int, y: Int): Int = x * y  
        else -> return fun(x: Int, y: Int): Int = 0  
    }  
}
```



Лямбда-выражения

Лямбда-выражение – это функция, записанная в виде выражения, и которую можно передавать как аргумент в другие функции.

```
{println("Hello Kotlin")}
```

```
fun main() {
```

```
    val hello = {println("Hello Kotlin")}
```

```
    hello()
```

```
    hello()
```

```
}
```

```
val hello: ()->Unit = {println("Hello Kotlin")}
```

```
fun main() {
```

```
    {println("Hello Kotlin")}() //можно запускать как обычную функцию
```

```
}
```




Лямбда-выражения

Передача параметров

```
fun main() {  
    val printer = {message: String -> println(message)}  
    printer("Hello")  
    printer("Good Bye")  
}
```

```
fun main() {  
    {message: String -> println(message)}("Welcome to Kotlin")  
}
```



Лямбда-выражения

Передача параметров

Если параметров несколько, то они передаются слева от стрелки через запятую:

```
fun main() {  
    val sum = {x:Int, y:Int -> println(x + y)}  
    sum(2, 3)  // 5  
    sum(4, 5)  // 9  
}
```



Лямбда-выражения

При выполнении нескольких действий можно размещать инструкции на отдельных строках после стрелки:

```
val sum = {x:Int, y:Int ->  
    val result = x + y  
    println("$x + $y = $result")  
}
```



Лямбда-выражения

Возвращение результата

```
val hello = { println("Hello") }
```

```
val h = hello()           // h представляет тип Unit
```

```
val printer = { message: String -> println(message) }
```

```
val p = printer("Welcome") // p представляет тип Unit
```



Лямбда-выражения

```
fun main() {  
    val sum = {x:Int, y:Int -> x + y}  
    val a = sum(2, 3)  // 5  
    val b = sum(4, 5)  // 9  
    println("a=$a b=$b")  
}
```



Лямбда-выражения

Если лямбда-выражение многострочное – возвращается то значение, которое генерируется последней инструкцией:

```
val sum = {x:Int, y:Int ->  
    val result = x + y  
    println("$x + $y = $result")  
    result  
}
```



Лямбда-выражения

Лямбда-выражения как аргументы функций

```
fun main() {  
    val sum = {x:Int, y:Int -> x + y }  
    doOperation(3, 4, sum)                // 7  
    doOperation(3, 4, {a:Int, b: Int -> a * b}) // 12  
}  
  
fun doOperation(x: Int, y: Int, op: (Int, Int) ->Int){  
    val result = op(x, y)  
    println(result)  
}
```



Лямбда-выражения

Возвращение лямбда-выражения из функции

```
fun main() {  
    val action1 = selectAction(1)  
    val result1 = action1(4, 5)  
    println(result1)    // 9  
  
    val action2 = selectAction(3)  
    val result2 = action2(4, 5)  
    println(result2)    // 20  
  
    val action3 = selectAction(9)  
    val result3 = action3(4, 5)  
    println(result3)    // 0  
}  
  
fun selectAction(key: Int): (Int, Int) -> Int {  
    // определение возвращаемого результата  
    when(key){  
        1 -> return {x, y -> x + y }  
        2 -> return {x, y -> x - y }  
        3 -> return {x, y -> x * y }  
        else -> return {x, y -> 0 }  
    }  
}
```




Спасибо за внимание!